

Lab-4: Applied Statistical Modeling & Interactive Web Dashboard

Course: Statistical Modeling with Python

Target Level: M.Sc. Data Science (Sem 1)

Total Weightage: 100 Marks

Suggested Tools: Python, statsmodels, Streamlit, VS Code

1. Assignment Overview & Learning Objectives

In this assignment, students will execute an end-to-end data science project spanning rigorous statistical analysis and real-world software delivery. Rather than terminating in a static Jupyter Notebook, students are required to construct an interactive, responsive web application using Streamlit. This ensures students develop strong fluency in translating formal mathematical models and inferential tests into actionable, user-centric data tools.

- Key Learning Outcomes:
 - Formulate and evaluate parametric and non-parametric statistical hypothesis tests.
 - Fit Ordinary Least Squares (OLS) regression models using statsmodels and interpret statistical coefficients.
 - Perform rigorous residual diagnostics to validate standard Gauss-Markov assumptions.
 - Architect and deploy a modern interactive data dashboard in Python using Streamlit.

2. Dataset Selection (Choose ONE)

Students may select any one of the following curated datasets or use an equivalent open-access tabular dataset containing both quantitative and categorical features:

- **Option A: Medical Insurance Costs (Recommended):** Features include age, sex, BMI, children, smoker, region, and medical charges. Ideal for examining non-linear relations and interaction terms between smoking and BMI.
- **Option B: Restaurant Tipping Behavior (tips dataset):** Features include total_bill, tip, sex, smoker, day, time, and size. Ideal for simple/multiple linear modeling and two-sample t-tests.
- **Option C: California / Ames Housing Subset:** Features include median income, housing age, rooms, population, and house value. Ideal for multi-variable regression, collinearity checks, and log-transformations.

3. Tasks & Technical Requirements

Part 1: Exploratory Data Analysis & Hypothesis Testing

- **Descriptive Metrics:** Calculate measures of central tendency (mean, median), dispersion (standard deviation, IQR), skewness, and kurtosis across numerical features.
- **Visual Exploration:** Generate distribution plots (histograms/KDE) and bivariate scatter plots with correlation matrices.
- **Hypothesis Test 1 (Compare 2 Groups):** Pick two groups (e.g., Smokers vs. Non-Smokers) and compare a continuous metric (e.g., Medical Charges).
 - Formulate H_0 (No difference between groups) and H_1 (There is a significant difference).
 - Check normality using Shapiro-Wilk and check equal variance using Levene's test.
 - If normal: run a Two-Sample t-test. If not normal: run a Mann-Whitney U test.
 - State your final conclusion at $\alpha = 0.05$ (Reject H_0 or Fail to Reject H_0).
- **Hypothesis Test 2 (Pick ONE):**
 - Option A (Chi-Square Test): Test if two categorical variables are linked (e.g., Is smoking status linked to region?).
 - Option B (One-Way ANOVA): Test if a numerical value differs across 3 or more groups (e.g., Do average charges differ across the 4 geographic regions?).

Part 2: Statistical Modeling & Diagnostic Checks

- **Model Formulation:** Specify a multiple linear regression model: $Y = \beta_0 + \beta_1 X_1 + \dots + \beta_k X_k + \epsilon$ using `statsmodels.api.OLS`.
- **Parameter Interpretation:** Interpret estimated coefficients (β), p-values, 95% confidence intervals, and the overall model fit (R^2 and Adjusted R^2).
- **Gauss-Markov Diagnostic Checks:**
 - Linearity & Homoscedasticity: Plot Residuals vs. Fitted values.
 - Normality of Residuals: Generate a Q-Q plot and report Jarque-Bera or Omnibus test results.
 - Multicollinearity: Calculate Variance Inflation Factor (VIF) for all continuous predictors.

Part 3: Interactive Streamlit Dashboard

The Streamlit application (`app.py`) must be organized into three distinct tabs or navigation pages:

- **Tab 1 — Data Exploration:** Include interactive sidebar filters (e.g., slider for age/bill ranges, category multi-select), reactive plots (Plotly or Seaborn), and dataset summary statistics.
- **Tab 2 — Hypothesis Testing Lab:** Dynamic dropdown menus allowing the user to select categorical factors and numerical metrics, automatically computing test statistics, p-values, and rendering clear conclusions (Reject / Fail to Reject H_0 at $\alpha = 0.05$).
- **Tab 3 — Live Prediction & Diagnostics:** Interactive user inputs (sliders/number boxes) generating real-time model predictions with a 95% confidence/prediction interval, followed by rendered residual diagnostic plots.

4. Development Environment Guidelines

Students are strongly encouraged to complete the assignment using VS Code (or a local IDE) to foster professional software engineering workflows. A standard Google Colab fallback workflow is permitted if local setup is constrained.

Method	Execution & Workflow Details
Local IDE (VS Code) — Recommended	1. Set up virtual environment: <code>python -m venv venv</code> 2. Install dependencies: <code>pip install -r requirements.txt</code> 3. Launch dashboard: <code>streamlit run app.py</code> (opens at localhost:8501)
Google Colab (Fallback)	1. Install: <code>!pip install streamlit</code> 2. Write code cell: <code>%%writefile app.py</code> 3. Tunnel using Localtunnel: <code>!streamlit run app.py & npx localtunnel --port 8501</code>

6. Submission Guidelines

- **GitHub Repository:** Provide a clean repository containing `app.py`, `requirements.txt`, and a `data/` folder (or fetch URL).
- **README.md Documentation:** Must include instructions to run the application, dataset summary, and a concise synthesis of statistical findings.
- **Optional Bonus (+5 Marks):** Deploy the completed dashboard to Streamlit Community Cloud (share.streamlit.io) and attach the live public URL in the repository header.